A1 – CFSD Practicals

Full Stack Development – Java & Web Technology

SE IT-A | SLRTCE | Roll No: 01 – Mohan

Experiment 1a: Rectangle Class with Object Creation

Aim:

Write a Java program to create a class Rectangle and its object. Use the main method to create an object of the Rectangle class and access its fields using the object.

Code:

```
public class Rectangle {
double length;
double width;

void displayArea() {
double area = length * width;
System.out.println("Area of Rectangle: " + area);
}

void displayPerimeter() {
double perimeter = 2 * (length + width);
System.out.println("Perimeter of Rectangle: " + perimeter);
}

public static void main(String[] args) {
Rectangle rect = new Rectangle();
rect.length = 10.5;
rect.width = 5.5;

System.out.println("Length: " + rect.length);
System.out.println("Width : " + rect.width);
rect.displayArea();
rect.displayPerimeter();
}
}
```

Output:

```
Length: 10.5
Width : 5.5
Area of Rectangle: 57.75
Perimeter of Rectangle: 32.0
```

Conclusion:

In this experiment, we successfully created a Java class named Rectangle with fields length and width. An object of the class was instantiated in the main method and its fields were accessed and modified directly using the dot operator. We also implemented methods to calculate and display the area and perimeter. This experiment demonstrated the concept of class definition, object creation, and accessing instance variables in Java.

Experiment 1b: Default and Parameterized Constructors

Aim:

Write a Java program to implement default and parameterized constructors.

Code:

```
public class Student {
String name;
int age;
String course;

// Default Constructor
Student() {
name = "Unknown";
age = 0;
course = "Not Assigned";
System.out.println("Default Constructor Called");
}

// Parameterized Constructor
Student(String name, int age, String course) {
this.name = name;
this.age = age;
this.course = course;
System.out.println("Parameterized Constructor Called");
}

void display() {
System.out.println("Name : " + name);
System.out.println("Age : " + age);
System.out.println("Course: " + course);
System.out.println("---------------------");
}

public static void main(String[] args) {
Student s1 = new Student();
s1.display();

Student s2 = new Student("Mohan", 20, "IT");
s2.display();
}
}
```

Output:

```
Default Constructor Called
Name : Unknown
Age : 0
Course: Not Assigned
---------------------
Parameterized Constructor Called
Name : Mohan
Age : 20
Course: IT
---------------------
```

Conclusion:

This experiment demonstrated the use of both default and parameterized constructors in Java. The default constructor initializes object fields with default values automatically when no arguments are provided, while the parameterized constructor allows users to supply specific values during object creation. Constructor overloading

was also observed. This is a fundamental concept in Object-Oriented Programming for efficient object initialization.

Experiment 2: Method Overloading using Inheritance in Java

Aim:

Write a program to implement Method Overloading using Inheritance in Java.

Code:

```
class Shape {
void area() {
System.out.println("Area method of Shape class (no args)");
}

void area(double side) {
System.out.println("Area of Square = " + (side * side));
}

void area(double length, double width) {
System.out.println("Area of Rectangle = " + (length * width));
}
}

class Circle extends Shape {
void area(double radius, String shape) {
double a = 3.14159 * radius * radius;
System.out.println("Area of Circle = " + a);
}
}

public class OverloadingDemo {
public static void main(String[] args) {
Circle c = new Circle();
c.area();
c.area(5.0);
c.area(4.0, 6.0);
c.area(7.0, "circle");
}
}
```

Output:

```
Area method of Shape class (no args)
Area of Square = 25.0
Area of Rectangle = 24.0
Area of Circle = 153.93791
```

Conclusion:

This experiment demonstrated method overloading in combination with inheritance in Java. The parent class Shape defined multiple overloaded versions of the area() method with different parameter lists. The child class Circle extended Shape and added its own overloaded variant. Method overloading improves code readability and allows the same method name to handle different types of input, which is a key principle of compile-time polymorphism in Java.

Experiment 2b: Method Overriding using Inheritance in Java

Aim:

Write a program to implement Method Overriding using Inheritance in Java.

Code:

```
class Animal {
String name;

Animal(String name) {
this.name = name;
}

void sound() {
System.out.println(name + " makes a generic animal sound.");
}

void display() {
System.out.println("Animal: " + name);
}
}

class Dog extends Animal {
Dog(String name) {
super(name);
}

// Overriding sound() method
@Override
void sound() {
System.out.println(name + " says: Woof! Woof!");
}
}

class Cat extends Animal {
Cat(String name) {
super(name);
}

// Overriding sound() method
@Override
void sound() {
System.out.println(name + " says: Meow! Meow!");
}
}

class Cow extends Animal {
Cow(String name) {
super(name);
}

// Overriding sound() method
@Override
void sound() {
System.out.println(name + " says: Moo! Moo!");
}
}

public class OverridingDemo {
public static void main(String[] args) {
Animal a = new Animal("Generic Animal");
a.sound();

Dog d = new Dog("Buddy");
d.sound();
```

```
Cat c = new Cat("Whiskers");
c.sound();

Cow cow = new Cow("Bessie");
cow.sound();

System.out.println("\n--- Using Polymorphism ---");
Animal[] animals = { new Dog("Rex"), new Cat("Luna"), new Cow("Daisy") };
for (Animal animal : animals) {
animal.sound(); // Runtime polymorphism
}
}
}
```

Output:

```
Generic Animal makes a generic animal sound.
Buddy says: Woof! Woof!
Whiskers says: Meow! Meow!
Bessie says: Moo! Moo!

--- Using Polymorphism ---
Rex says: Woof! Woof!
Luna says: Meow! Meow!
Daisy says: Moo! Moo!
```

Conclusion:

This experiment successfully demonstrated method overriding in Java using inheritance. The parent class Animal defined a sound() method which was overridden in each subclass (Dog, Cat, Cow) with specific behaviour. The @Override annotation ensures compile-time checking. When animal objects were referenced through a parent class reference array, Java's runtime polymorphism (dynamic method dispatch) automatically called the correct overridden method of the actual object at runtime. This is a core principle of runtime polymorphism in Object-Oriented Programming.

Experiment 3a: List Interface – ArrayList Operations

Aim:

Write a Java program to implement the List interface and perform basic operations such as adding elements to a list.

Code:

```
import java.util.ArrayList;
import java.util.List;

public class ListDemo {
public static void main(String[] args) {
List&lt;String&gt; fruits = new ArrayList&lt;&gt;();

// Adding elements
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");
fruits.add("Orange");
fruits.add(2, "Grapes"); // Add at index

System.out.println("List after adding elements: " + fruits);
System.out.println("Element at index 1: " + fruits.get(1));
```

```
System.out.println("Size of list: " + fruits.size());

// Remove element
fruits.remove("Banana");
System.out.println("After removal: " + fruits);

// Iterate
System.out.println("Iterating using for-each:");
for (String fruit : fruits) {
System.out.println(" " + fruit);
}
}
}
```

Output:

```
List after adding elements: [Apple, Banana, Grapes, Mango, Orange]
Element at index 1: Banana
Size of list: 5
After removal: [Apple, Grapes, Mango, Orange]
Iterating using for-each:
Apple
Grapes
Mango
Orange
```

Conclusion:

In this experiment, we successfully implemented the List interface in Java using the ArrayList class. We performed basic list operations such as adding elements at the end and at a specific index, accessing elements by index, finding the list size, removing elements, and iterating over the list. The List interface provides an ordered, index-based collection that allows duplicate elements, making it highly versatile for real-world Java applications.

Experiment 3b: Set and Map Interface – HashSet and HashMap

Aim:

Write a Java program to implement the Set and Map interface using HashSet and HashMap and perform basic operations such as adding elements.

Code:

```
import java.util.HashSet;
import java.util.HashMap;
import java.util.Set;
import java.util.Map;

public class SetMapDemo {
public static void main(String[] args) {
// HashSet Demo
Set&lt;String&gt; citySet = new HashSet&lt;&gt;();
citySet.add("Mumbai");
citySet.add("Pune");
citySet.add("Delhi");
citySet.add("Mumbai"); // Duplicate – won't be added
System.out.println("HashSet: " + citySet);
System.out.println("Contains Pune: " + citySet.contains("Pune"));
citySet.remove("Delhi");
System.out.println("After remove: " + citySet);

System.out.println();

// HashMap Demo
Map&lt;Integer, String&gt; studentMap = new HashMap&lt;&gt;();
studentMap.put(1, "Mohan");
studentMap.put(2, "Rahul");
studentMap.put(3, "Priya");
System.out.println("HashMap: " + studentMap);
System.out.println("Student with ID 2: " + studentMap.get(2));
studentMap.remove(3);
System.out.println("After remove: " + studentMap);

System.out.println("Iterating HashMap:");
for (Map.Entry&lt;Integer,String&gt; entry : studentMap.entrySet()) {
System.out.println(" ID: " + entry.getKey() + " | Name: " + entry.getValue());
}
}
}
```

Output:

```
HashSet: [Delhi, Pune, Mumbai]
Contains Pune: true
After remove: [Pune, Mumbai]

HashMap: {1=Mohan, 2=Rahul, 3=Priya}
Student with ID 2: Rahul
After remove: {1=Mohan, 2=Rahul}
Iterating HashMap:
ID: 1 | Name: Mohan
ID: 2 | Name: Rahul
```

Conclusion:

This experiment demonstrated the use of the Set and Map interfaces from the Java Collections Framework. HashSet stores unique elements with no defined order and does not allow duplicates. HashMap stores key-value pairs, providing fast lookup by key. Both are backed by hashing mechanisms for O(1) average-case performance. These collection types are widely used in real-world applications for data storage, lookup, and deduplication tasks.

Experiment 4: Multithreading – Thread Class and Runnable Interface

Aim:

Write a Java program to illustrate multithreading by implementing threads using: a) Thread class b) Runnable Interface.

a) Using Thread Class:

```
class MyThread extends Thread {
String threadName;

MyThread(String name) {
this.threadName = name;
}

public void run() {
for (int i = 1; i &lt;= 3; i++) {
System.out.println(threadName + " - Count: " + i);
try { Thread.sleep(300); } catch (InterruptedException e) {}
}
}
}

public class ThreadDemo {
public static void main(String[] args) {
MyThread t1 = new MyThread("Thread-A");
MyThread t2 = new MyThread("Thread-B");
t1.start();
t2.start();
}
}
```

b) Using Runnable Interface:

```
class MyRunnable implements Runnable {
String taskName;

MyRunnable(String name) {
this.taskName = name;
}

public void run() {
for (int i = 1; i &lt;= 3; i++) {
System.out.println(taskName + " - Step: " + i);
try { Thread.sleep(200); } catch (InterruptedException e) {}
}
}
}

public class RunnableDemo {
public static void main(String[] args) {
Thread t1 = new Thread(new MyRunnable("Task-1"));
```

```
Thread t2 = new Thread(new MyRunnable("Task-2"));
t1.start();
t2.start();
}
}
```

Output (may vary due to thread scheduling):

```
Thread-A - Count: 1
Thread-B - Count: 1
Thread-A - Count: 2
Thread-B - Count: 2
Thread-A - Count: 3
Thread-B - Count: 3
```

Conclusion:

This experiment illustrated two ways to implement multithreading in Java. The Thread class approach requires extending Thread and overriding the run() method, while the Runnable interface approach allows implementing the Runnable interface and passing it to a Thread object. Using Runnable is preferred as Java does not support multiple inheritance, so implementing Runnable allows the class to extend another class simultaneously. Both approaches enable concurrent execution of tasks, improving application performance and responsiveness.

Experiment 5a: Responsive Login Form using HTML5, CSS3, Bootstrap and JavaScript

Aim:

Write a program to develop a responsive login form using HTML5, CSS3 and Bootstrap and validate user input using JavaScript and DOM manipulation technique.

Code (login.html):

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
&lt;meta charset="UTF-8"&gt;
&lt;meta name="viewport" content="width=device-width, initial-scale=1"&gt;
&lt;title&gt;Login Form&lt;/title&gt;
&lt;link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet"&gt;
&lt;style&gt;
body { background: linear-gradient(135deg, #1a237e, #42a5f5); min-height: 100vh;
display: flex; align-items: center; justify-content: center; }
.card { border-radius: 16px; box-shadow: 0 8px 32px rgba(0,0,0,0.3); }
.btn-login { background: #1a237e; border: none; }
.btn-login:hover { background: #0d47a1; }
.error { color: red; font-size: 0.8rem; }
&lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
&lt;div class="card p-4" style="width:380px"&gt;
&lt;h3 class="text-center mb-3"&gt;Login&lt;/h3&gt;
&lt;div class="mb-3"&gt;
&lt;label&gt;Email&lt;/label&gt;
&lt;input type="text" id="email" class="form-control" placeholder="Enter email"&gt;
&lt;span class="error" id="emailErr"&gt;&lt;/span&gt;
&lt;/div&gt;
&lt;div class="mb-3"&gt;
&lt;label&gt;Password&lt;/label&gt;
&lt;input type="password" id="password" class="form-control" placeholder="Enter password"&gt;
&lt;span class="error" id="passErr"&gt;&lt;/span&gt;
&lt;/div&gt;
&lt;button class="btn btn-login text-white w-100" onclick="validate()"&gt;Login&lt;/button&gt;
&lt;div id="successMsg" class="alert alert-success mt-3 d-none"&gt;Login Successful!&lt;/div&gt;
&lt;/div&gt;
&lt;script&gt;
function validate() {
let email = document.getElementById('email').value;
let password = document.getElementById('password').value;
let valid = true;

document.getElementById('emailErr').textContent = '';
document.getElementById('passErr').textContent = '';
document.getElementById('successMsg').classList.add('d-none');

if (email === '' || !email.includes('@')) {
document.getElementById('emailErr').textContent = 'Enter a valid email.';
valid = false;
}
if (password.length &lt; 6) {
document.getElementById('passErr').textContent = 'Password must be at least 6 chars.';
```

```
valid = false;
}
if (valid) {
document.getElementById('successMsg').classList.remove('d-none');
}
}
&lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Conclusion:

In this experiment, we developed a responsive login form using HTML5 for structure, CSS3 for styling, Bootstrap 5 for responsive layout, and JavaScript for client-side validation using DOM manipulation. The form validates user inputs in real-time, checking for a valid email format and minimum password length. Error messages are dynamically displayed using DOM manipulation without any page reload. This experiment demonstrated the integration of front-end technologies for building interactive, user-friendly web forms.

Experiment 5b: Interactive HTML Form with Client-Side Validation using JavaScript DOM

Aim:

Write a program to develop and an interactive HTML form and implement client-side validation using JavaScript with DOM manipulation technique.

Code (registration.html):

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
&lt;meta charset="UTF-8"&gt;
&lt;title&gt;Registration Form&lt;/title&gt;
&lt;style&gt;
* { box-sizing: border-box; font-family: Arial, sans-serif; }
body { background: #f0f4f8; display: flex; justify-content: center; padding: 40px; }
.form-box { background: #fff; padding: 30px; border-radius: 10px;
box-shadow: 0 4px 16px rgba(0,0,0,0.1); width: 420px; }
h2 { text-align: center; color: #1a237e; }
label { font-weight: bold; font-size: 0.9rem; }
input, select { width: 100%; padding: 8px; margin: 4px 0 2px;
border: 1px solid #ccc; border-radius: 6px; }
.error { color: #c62828; font-size: 0.78rem; }
button { width: 100%; padding: 10px; background: #1a237e; color: white;
border: none; border-radius: 6px; cursor: pointer; margin-top: 12px; }
button:hover { background: #0d47a1; }
#result { text-align: center; margin-top: 12px; color: green; font-weight: bold; }
&lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
&lt;div class="form-box"&gt;
&lt;h2&gt;Registration&lt;/h2&gt;
&lt;label&gt;Full Name&lt;/label&gt;
&lt;input type="text" id="name" placeholder="Full Name"&gt;
&lt;span class="error" id="nameErr"&gt;&lt;/span&gt;

&lt;label&gt;Email&lt;/label&gt;
&lt;input type="text" id="email" placeholder="Email"&gt;
```

```
&lt;span class="error" id="emailErr"&gt;&lt;/span&gt;

&lt;label&gt;Phone&lt;/label&gt;
&lt;input type="text" id="phone" placeholder="10-digit phone"&gt;
&lt;span class="error" id="phoneErr"&gt;&lt;/span&gt;

&lt;label&gt;Gender&lt;/label&gt;
&lt;select id="gender"&gt;
&lt;option value=""&gt;-- Select --&lt;/option&gt;
&lt;option&gt;Male&lt;/option&gt;
&lt;option&gt;Female&lt;/option&gt;
&lt;option&gt;Other&lt;/option&gt;
&lt;/select&gt;
&lt;span class="error" id="genderErr"&gt;&lt;/span&gt;

&lt;button onclick="validateForm()"&gt;Register&lt;/button&gt;
&lt;div id="result"&gt;&lt;/div&gt;
&lt;/div&gt;
&lt;script&gt;
function validateForm() {
let name = document.getElementById('name').value.trim();
let email = document.getElementById('email').value.trim();
let phone = document.getElementById('phone').value.trim();
let gender = document.getElementById('gender').value;
let valid = true;

['nameErr','emailErr','phoneErr','genderErr'].forEach(id =&gt;
document.getElementById(id).textContent = '');
document.getElementById('result').textContent = '';

if (name.length &lt; 3) {
document.getElementById('nameErr').textContent = 'Name must be at least 3 characters.';
valid = false;
}
if (!email.match(/^[^\s@]+@[^\s@]+\.[^\s@]+$/)) {
document.getElementById('emailErr').textContent = 'Enter a valid email.';
valid = false;
}
if (!phone.match(/^\d{10}$/)) {
document.getElementById('phoneErr').textContent = 'Enter a valid 10-digit phone.';
valid = false;
}
if (gender === '') {
document.getElementById('genderErr').textContent = 'Please select a gender.';
valid = false;
}
if (valid) {
document.getElementById('result').textContent =
'Registration Successful! Welcome, ' + name + '!';
}
}
&lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Conclusion:

This experiment demonstrated building an interactive registration form in HTML with comprehensive client-side validation using JavaScript DOM manipulation. The form validates multiple fields including name length, email

format using regex, 10-digit phone number format, and gender selection. Error messages are injected dynamically into the DOM without any page reload, providing immediate feedback to the user. This approach enhances user experience and reduces unnecessary server requests by catching errors on the client side first.

Experiment 7: Connect Spring Boot Application to MySQL using Spring Data JPA and Perform CRUD Operations

Aim:

Connect a Spring Boot application to MySQL using Spring Data JPA and perform CRUD (Create, Read, Update, Delete) operations.

application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Entity – Student.java:

```
import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;
private String name;
private String email;
private String course;

// Getters and Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getCourse() { return course; }
public void setCourse(String course) { this.course = course; }
}
```

Repository – StudentRepository.java:

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository&lt;Student, Integer&gt; {
}
```

Controller – StudentController.java:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

@Autowired
private StudentRepository repo;
```

```
// CREATE
@PostMapping
public Student addStudent(@RequestBody Student s) {
return repo.save(s);
}

// READ ALL
@GetMapping
public List&lt;Student&gt; getAllStudents() {
return repo.findAll();
}

// READ BY ID
@GetMapping("/{id}")
public Student getStudentById(@PathVariable int id) {
return repo.findById(id).orElse(null);
}

// UPDATE
@PutMapping("/{id}")
public Student updateStudent(@PathVariable int id, @RequestBody Student updated) {
updated.setId(id);
return repo.save(updated);
}

// DELETE
@DeleteMapping("/{id}")
public String deleteStudent(@PathVariable int id) {
repo.deleteById(id);
return "Student with ID " + id + " deleted.";
}
}
```

API Testing (Postman / curl):

```
POST http://localhost:8080/students
Body: {"name":"Mohan","email":"mohan@example.com","course":"IT"}

GET http://localhost:8080/students
GET http://localhost:8080/students/1

PUT http://localhost:8080/students/1
Body: {"name":"Mohan Updated","email":"mohan@example.com","course":"IT"}

DELETE http://localhost:8080/students/1
```

Conclusion:

In this experiment, we successfully connected a Spring Boot application to a MySQL database using Spring Data JPA. We created a Student entity mapped to a database table, defined a JPA Repository interface to handle database operations, and exposed RESTful API endpoints using a REST Controller for all four CRUD operations. Spring Data JPA significantly reduces boilerplate code by providing built-in methods such as save(), findAll(), findById(), and deleteById(). This experiment demonstrated a complete backend development workflow using the Spring ecosystem.